

Tarea 1: Computación de Alto Rendimiento (IIC3533)

Josefina Abbott Cerda, José Ignacio Salas Coeymans y Borja Smith Vergara

11 de septiembre de 2026

(a) Generación de datos sintéticos

Para generar los datos sintéticos, seguimos los siguientes pasos:

1. Generar los datos, usando el generador aleatorio de NumPy (`np.random.default_rng`) fijando una semilla base, nosotros la establecimos como 1111.
2. Generar el vector de coeficientes reales $\beta^* \in \mathbb{R}^{301}$ muestreando de una distribución normal estándar $\mathcal{N}(0, 1)$.
3. Crear una matriz base de 10000×300 con valores independientes $\mathcal{N}(0, 1)$.
4. Concatenar horizontalmente un vector columna de unos a la matriz base para incorporar el intercepto (β_0), obteniendo así la matriz de diseño $X \in \mathbb{R}^{10000 \times 301}$.
5. Calcular el vector de respuesta $y \in \mathbb{R}^{10000}$ con el producto matricial $y = X\beta^* + \varepsilon$, donde ε es un vector de ruido con 10000 muestras i.i.d. de $\mathcal{N}(0, 1)$.

Para correr el script de generación de datos, se puede ejecutar el siguiente comando en la terminal:

```
python src/data_generation.py
```

(b) Implementaciones de bootstrapping

Evolución de `bs_auto.py`

1. Cambios implementados por versión

- **Versión 1 (Ingenua):** Utiliza la configuración por defecto de scikit-learn. Emplea `LinearRegression()` (que calcula un intercepto internamente) y extrae los coeficientes iterando con un ciclo `for` estándar y `.append()`.
- **Versión 2:** Introduce `fit_intercept=False`. Dado que la matriz de diseño X ya incluye una columna de unos pre-concatenada[cite: 1], esto evita el cálculo de un intercepto redundante.

- **Versión 3:** Reemplaza el ciclo `for` por una comprensión de listas (*list comprehension*) vectorizada para la extracción de coeficientes, mejorando la asignación de memoria.
- **Versión 4:** Añade explícitamente el parámetro `bootstrap=True` al `BaggingRegressor` para garantizar la correctitud del algoritmo (muestreo con reemplazo)[cite: 1], manteniendo la comprensión de listas.
- **Versión 5:** Mantiene los parámetros óptimos del modelo, pero revierte la extracción de coeficientes al ciclo `for` con `.append()` para contrastar el manejo de memoria final.

2. Tiempos de ejecución

Versión	Características Principales	Tiempo ($p = 1$)
v1	Intercepto activo, ciclo for	4.7474 s
v2	Sin intercepto, ciclo for	5.1836 s
v3	Sin intercepto, list comprehension	5.3599 s
v4	Sin inter., list comp., bootstrap explícito	4.2048 s
v5	Sin inter., ciclo for, bootstrap explícito	4.1570 s

Cuadro 1: Tiempos de ejecución empíricos para distintas iteraciones de `bs_auto.py`.

3. Análisis de la versión óptima

La **Versión 4** es computacional y estructuralmente la más óptima.

En primer lugar, al definir `fit_intercept=False`, se elimina una operación matricial redundante que scikit-learn realizaría en cada uno de los $B = 48$ remuestreos, ya que la matriz X construida ya contempla el intercepto en su primera columna[cite: 1]. Además, declarar `bootstrap=True` asegura el rigor matemático del remuestreo exigido[cite: 1], protegiendo el código contra posibles cambios en los valores por defecto de la librería.

Si bien empíricamente la Versión 5 registró un tiempo marginalmente inferior en esta ejecución específica (4.15s vs 4.20s), esta diferencia de ~ 0.05 segundos es atribuible a la varianza natural del *scheduling* de la CPU en el sistema operativo. A nivel de arquitectura de software, la Versión 4 es superior porque utiliza una comprensión de listas (`[est.coef_ for est in ...]`) en lugar de un `.append()` dinámico. Esto permite que el motor en C de Python asigne el bloque de memoria necesario de una sola vez, en lugar de redimensionar la lista continuamente en cada iteración, previniendo cuellos de botella en la memoria RAM a medida que el proceso escala.

Para correr las implementaciones, se pueden ejecutar los siguientes comandos en la terminal:

```
python src/bs_auto.py
python src/bs_sklearn.py
python src/bs_numpy.py
```

- (c) Correctitud y reproducibilidad
- (d) Backend multiprocessing de joblib
- (e) Actividad del sistema y oversubscription
- (f) Tiempos de ejecución en función de p
- (g) Speedup $S(p)$ y Eficiencia $E(p)$
- (h) Overhead $T_o(p)$
- (i) Variación de procesos y threads internos
- (j) Comparación entre computadores

Declaración uso de IA